



Especialización en Diseño y Desarrollo de Videojuegos

Facultad de ingeniería

ACA 2: mplementar en Lua sobre el motor Love 2D un videojuego tipo Breakout completamente jugable, partiendo del diseño orientado a objetos elaborado en el ACA 1.

Presentado por:

Jhoan Andrés Castelblanco Utria

Julian Eleicer Orellanos

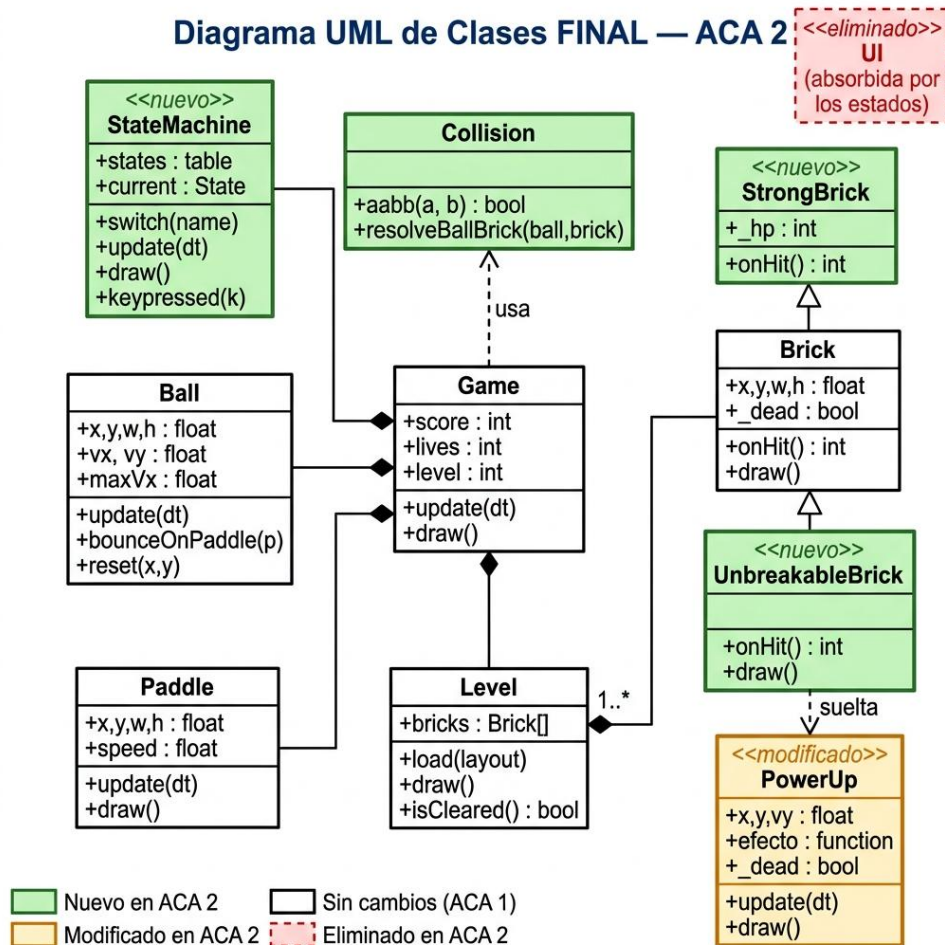
Bogotá, Cundinamarca 26 de julio de 2026

Repositorio Github ACA 2: <https://github.com/johancastel/Breakout-Guardians-of-the-Forge>

Repositorio Github Juego final: [https://github.com/johancastel/01\\_Breakout-Guardians-of-the-Forge\\_Final\\_Version](https://github.com/johancastel/01_Breakout-Guardians-of-the-Forge_Final_Version)

## 1. Diagrama UML de clases Vs la versión anterior

El delta en relación con el ACA1 es donde se agregan, modifica y adoben a las siguientes clases;



### - Clases y módulos agregados al juego:

**StateMachine (Clase):** Encargada de administrar el estado del juego. En el ACA 1 se había planteado un flujo lineal con banderas booleanas (isPaused, isGameOver). Esta máquina de estados permite separar la lógica de las pantallas (Título, Saque, Juego Activo, Pausa, Fin de Juego y Victoria).

**Collision (Módulo):** Se creó este módulo para centralizar la detección física AABB (Axis-Aligned Bounding Box) y la resolución de rebotes por el eje de menor solape.

**StrongBrick (Clase):** Subclase que hereda de Brick. Se agregó para representar bloques resistentes que requieren dos impactos del *Energy Core* (bola) para destruirse.

**UnbreakableBrick (Clase):** Subclase que hereda de Brick para representar bloques grises indestructibles.

PowerUp (Clase): Incorpora poderes flotantes que caen al destruir ciertos bloques. Se diseñó de forma genérica pasando una función callback en su constructor.

- Clases y Estructuras Modificadas

**Game:** Pasó de ser un controlador plano a ser un contenedor de estado compartido del mundo (composición). Almacena las instancias activas de Paddle, Ball, Level y la máquina de estados StateMachine (Game.sm), delegando el ciclo principal.

Level: Se modificó para que sus atributos y métodos coincidan con la lectura dirigida por datos. Ahora contiene una lista de objetos Brick y el método load (layout) para interpretar matrices bidimensionales.

- Elementos Eliminados

Se eliminaron interfaces redundantes y dependencias globales circulares que existían en el boceto preliminar.

## 2. Explicación del sistema de colisiones: cómo detectas (AABB)

El sistema de colisiones del juego está dividido en dos mecanismos distintos: uno para la colisión bola–bloque (gestionado por el módulo Collision) y otro para la colisión bola–paleta (gestionado directamente por el método Ball:bounceOnPaddle()).

La detección se basa en el algoritmo AABB, que verifica si dos rectángulos alineados con los ejes se solapan. La condición matemática es:

Dos rectángulos A y B se solapan si y solo si:

$$A.x < B.x + B.w \text{ AND } B.x < A.x + A.w \text{ AND } A.y < B.y + B.h \text{ AND } B.y < A.y + A.h$$

En el código de collision.lua esto se implementa así:

```
lua
function M.aabb(a, b)
    return a.x < b.x + b.w and b.x < a.x + a.w
    and a.y < b.y + b.h and b.y < a.y + a.h
end
```

Cada fotograma, el estado play.lua itera sobre todos los bloques activos y consulta M.aabb(ball, brick) para cada uno. Si hay colisión, se llama a M.resolveBallBrick().

Una vez detectada la colisión, el sistema debe decidir por qué lado chocó la bola para invertir la velocidad correcta (vx o vy). Para eso calcula el solape en cada eje y elige el de menor valor:

```

function M.resolveBallBrick(ball, brick)

  local ox = math.min(ball.x+ball.w, brick.x+brick.w)
    - math.max(ball.x, brick.x) -- solape en X
  local oy = math.min(ball.y+ball.h, brick.y+brick.h)
    - math.max(ball.y, brick.y) -- solape en Y

  if ox < oy then      -- chocó de LADO (eje X es menor)
    ball.vx = -ball.vx
    ball.x = ball.x + (ball.x < brick.x and -ox or ox)
  else                -- chocó ARRIBA / ABAJO (eje Y es menor)
    ball.vy = -ball.vy
    ball.y = ball.y + (ball.y < brick.y and -oy or oy)
  end
end

```

La reubicación inmediata ( $\text{ball.x} = \text{ball.x} \pm \text{ox}$ ) es clave: saca la bola del interior del bloque antes de continuar la simulación.

La colisión con la paleta usa una lógica diferente donde el ángulo de salida depende del punto exacto de impacto en la paleta, dándole al jugador control sobre la dirección de la bola.

```

function Ball:bounceOnPaddle(paddle)

  self.y = paddle.y - self.h      -- reubica encima de la paleta
  self.vy = -math.abs(self.vy)    -- garantiza que salga HACIA ARRIBA

  local centroBola = self.x + self.w / 2
  local centroPaleta = paddle.x + paddle.w / 2
  local t = (centroBola - centroPaleta) / (paddle.w / 2) -- rango: -1 .. +1

```

```

        self.vx = t * self.maxVx      -- ángulo proporcional al punto de
    impacto
end

```

### 3. Evidencia de POO

Lua no tiene clases nativas, pero implementa los tres pilares de la POO mediante metatables. A continuación se documenta cada pilar con fragmentos reales del código fuente.

#### - Encapsulación

La encapsulación agrupa datos y comportamiento dentro de una misma unidad, ocultando el estado interno, con metatables que vinculan datos y métodos en un mismo objeto, En brick.lua, el atributo `_dead` (convención de privado con `_`) nunca es modificado directamente desde afuera: el único canal autorizado es el método `onHit()`:

#### - Herencia

`StrongBrick` y `UnbreakableBrick` heredan de `Brick` usando la cadena de metatables de Lua, si un método no existe en la subclase, Lua lo busca automáticamente en la clase padre a través de `__index`.

#### - Polimorfismo

En el bucle de colisiones de `play.lua`. El estado `play` llama a `b:onHit()` sobre cada bloque sin importar si es `Brick`, `StrongBrick` o `UnbreakableBrick`.

| Tipo de bloque   | Comportamiento de onHit()                              | Puntos                                 |
|------------------|--------------------------------------------------------|----------------------------------------|
| Brick            | Se destruye en 1 golpe ( <code>_dead = true</code> )   | 10 pts                                 |
| StrongBrick      | Descuenta <code>_hp</code> ; se destruye al llegar a 0 | 10 pts por golpe, 20 pts al destruirse |
| UnbreakableBrick | No hace nada ( <code>_dead</code> nunca cambia)        | 0 pts                                  |

### 4. Estructura de los datos

El sistema de niveles sigue un patrón dirigido por datos, la geometría de cada nivel se define como una simple matriz de enteros en `levels.lua`, completamente separada de la lógica de

construcción que vive en `level.lua`. Esto hace que agregar un nuevo nivel sea tan sencillo como añadir una nueva tabla de números.

Cada nivel es una tabla de tablas (matriz 2D) donde cada entero es un código que representa un tipo de bloque y la visualización directa de cada fila de la matriz corresponde a una fila de bloques en pantalla, lo que permite editar o diseñar un nivel simplemente mirando los números, y no solo definir la rigurosidad de las filas, sino también el nivel de impacto de cada bloque, es decir podríamos hacer bloques sencillo, doble impacto e indestructibles.

La clase `Level` es la que interpreta esa matriz y la convierte en una lista plana de objetos instanciados. El método `load(layout)` recorre la matriz con dos bucles anidados y usa un `factory switch` (estructura `if/elseif`) para instanciar la clase correcta.

Para saber cuándo el jugador completa el nivel, `Level:isCleared()` recorre la lista y verifica que no quede ningún bloque vivo que no sea indestructible. Los bloques `UnbreakableBrick` nunca cuentan para la condición de victoria.

## 5. Reflexión final

El cambio más significativo fue pasar de un diseño teórico a una implementación funcional real, y esa transición nos llevó a tomar varias decisiones que no habíamos anticipado en el ACA 1.

La clase `UI` que habíamos modelado resultó ser innecesaria al implementar la máquina de estados (`StateMachine`), cada estado (`title`, `play`, `gameover`, etc.) se convirtió naturalmente en su propia pantalla con su propia lógica de dibujo. La información de vidas, puntaje y nivel quedó directamente en `play.lua`, eliminando una capa de abstracción que añadía complejidad sin beneficio real.

El diseño de `Brick` también evolucionó. En el ACA 1 lo habíamos planteado como una clase monolítica con un atributo `type` (tipo de bloque) y un condicional interno. En la implementación decidimos aplicar herencia real: `StrongBrick` y `UnbreakableBrick` son subclases independientes que sobrescriben `onHit()`. Esto eliminó todos los condicionales de tipo y demostró en la práctica la ventaja del polimorfismo: el bucle de colisiones no necesita saber con qué tipo de bloque está tratando.

Adicionalmente el módulo `Collision` no existía en el diseño del ACA 1. La lógica de colisiones estaba dispersa dentro de `Ball` y `Brick`. Aislarla en un módulo utilitario independiente fue la decisión más importante de esta entrega, porque permitió probar, leer y ajustar la física sin tocar las clases de entidad.